

Tablice Systolityczne i SIMD

Architektury Komputerów – Wykład 9

Kuba Nowak

Uniwersytet Wrocławski

13 kwietnia 2025

Computer Architecture

Lecture 7: VLIW, Systolic Arrays, Decoupled Access/Execute

Author: Prof. Onur Mutlu
(modified by Krystian Balcwski)

ETH Zürich
Spring 2021
7 May 2021

Computer Architecture

Lecture 10: Memory Organization & RowHammer

Author: Prof. Onur Mutlu
(modified by Krystian Balcwski)

ETH Zürich
Spring 2021
21 May 2021

Computer Architecture

Lecture 8: SIMD Processors

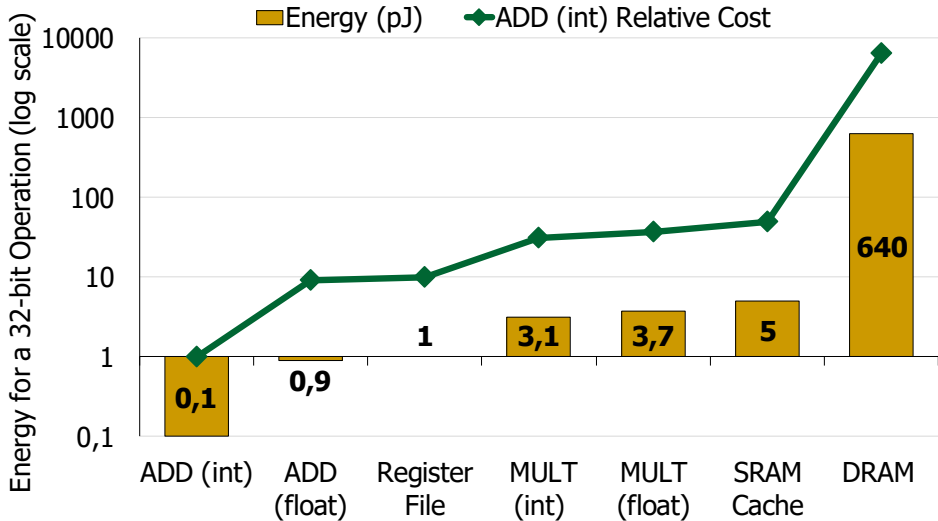
Author: Prof. Onur Mutlu
(modified by Krystian Balcwski)

ETH Zürich
Spring 2021
14 May 2021

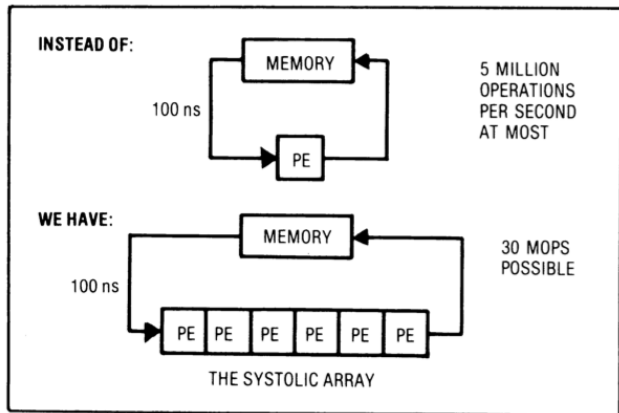
Zrównoleganie wykonania instrukcji

- Potokowanie
- Superskalarność
- Procesory dataflow
- Wykonanie out-of-order
- VLIW
- Tablice systolityczne (skurczowe)
- SIMD
- Droбноziarnista wielowątkowość

Data Movement vs. Computation Energy



Systolic Arrays



Memory: heart
Data: blood
PEs: cells

Memory pulses
data through
PEs

Figure 1. Basic principle of a systolic system.

- H. T. Kung, "Why Systolic Architectures?," IEEE Computer 1982.

Why Systolic Architectures?

- Idea: Data flows from the computer memory in a rhythmic fashion, passing through many processing elements before it returns to memory
- Similar to blood flow: heart $\rightarrow$ many cells $\rightarrow$ heart
 - Different cells “process” the blood
 - Many veins operate simultaneously
 - Can be many-dimensional
- Why? Special purpose accelerators/architectures need
 - Simple, regular design (keep # unique parts small and regular)
 - High concurrency $\rightarrow$ high performance
 - Balanced computation and I/O (memory) bandwidth

Systolic Architectures

- Basic principle: Replace a single PE with a **regular array of PEs** and **carefully orchestrate flow of data** between the PEs
 - Balance computation and memory bandwidth

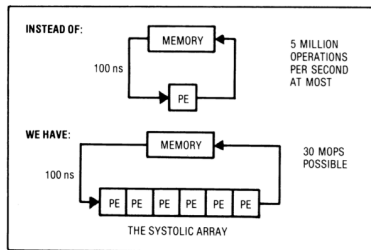


Figure 1. Basic principle of a systolic system.

- Differences from pipelining:
 - These are individual PEs
 - Array structure can be non-linear and multi-dimensional
 - PE connections can be multidirectional (and different speed)
 - PEs can have local memory and execute kernels (rather than a piece of the instruction)

Systolic Computation Example

■ Convolution

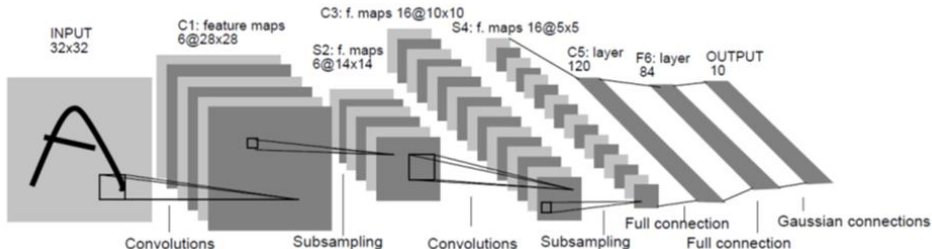
- Used in filtering, pattern matching, correlation, polynomial evaluation, etc ...
- Many **image processing** tasks
- **Machine learning**: up to hundreds of **convolutional layers** in Convolutional Neural Networks (CNN)

Given the sequence of weights $\{w_1, w_2, \dots, w_k\}$
and the input sequence $\{x_1, x_2, \dots, x_n\}$,
compute the result sequence $\{y_1, y_2, \dots, y_{n+1-k}\}$
defined by

$$y_i = w_1x_i + w_2x_{i+1} + \dots + w_kx_{i+k-1}$$

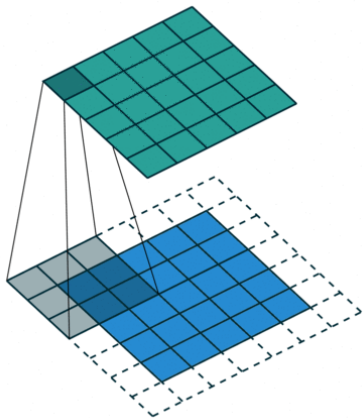
LeNet-5, a Convolutional Neural Network for Hand-Written Digit Recognition

This is a 1024*8 bit input, which will have a truth table of 2^{8196} entries



An Example of 2D Convolution

Output feature map



Input feature map

Structure information

Input: 5*5 (blue)

Kernel (filter): 3*3 (grey)

Output: 5*5 (green)

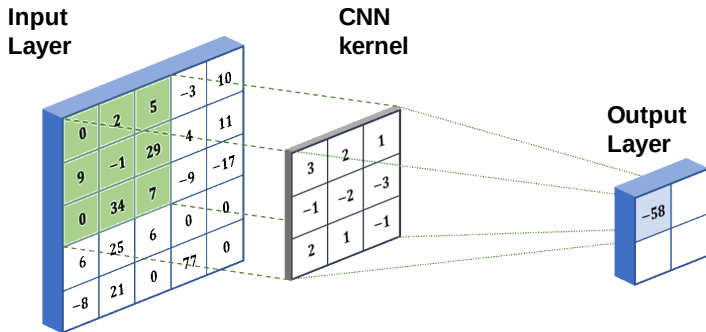
Computation information

Stride: 1

Padding: 1 (white)

Output Dim = (Input + 2*Padding
- Kernel) / Stride + 1

An Example of 2D Convolution



Convolutional Neural Networks: Demo

[Back to Yann's Home Publications](#)

LeNet-5 Demos

Unusual Patterns

[unusual styles](#)
[weirdos](#)

Invariance

[translation](#) (anim)
[scale](#) (anim)
[rotation](#) (anim)
[squeezing](#) (anim)
[stroke width](#) (anim)

Noise

[Resistance](#)
[noisy 3 and 6](#)
[noisy 2](#) (anim)
[noisy 4](#) (anim)

Multiple Character

[various stills](#)
[dancing 00](#) (anim)
[dancing 384](#) (anim)

Complex cases

(anim)
[35 -> 63](#)
[12 -> 4 -> 21](#)
[23 -> 32](#)
[30 + noise](#)
[31-51-57-61](#)

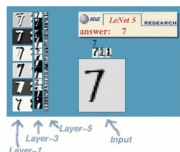
LeNet-5, convolutional neural networks

Convolutional Neural Networks are a special kind of multi-layer neural networks. Like almost every other neural networks they are trained with a version of the back-propagation algorithm. Where they differ is in the architecture.

Convolutional Neural Networks are designed to recognize visual patterns directly from pixel images with minimal preprocessing.

They can recognize patterns with extreme variability (such as handwritten characters), and with robustness to distortions and simple geometric transformations.

LeNet-5 is our latest convolutional network designed for handwritten and machine-printed character recognition. Here is an example of LeNet-5 in action.



Many more examples are available in the column on the left:

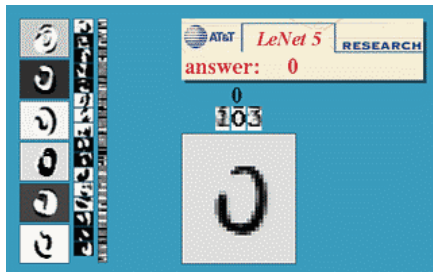
Several papers on LeNet and convolutional networks are available on my [publication page](#):

[LeCun et al., 1998]

Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner.
Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, november 1998.
[ps.gz](#)

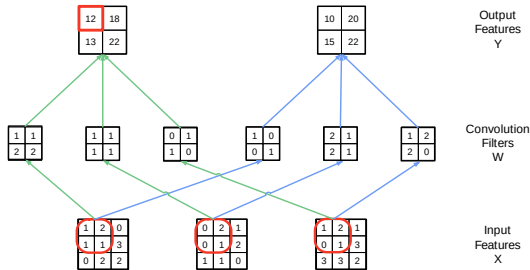
[Bottou et al., 1997]

L. Bottou, Y. LeCun, and Y. Bengio. Global training of



<http://yann.lecun.com/exdb/lenet/index.html>

Implementing a Convolutional Layer with Matrix Multiplication



$$\begin{array}{c}
 \begin{bmatrix} 1 & 1 & 2 & 2 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 2 & 1 & 2 & 1 & 1 & 2 & 2 & 0 \end{bmatrix} \\
 \text{Convolution Filters } W'
 \end{array}
 *
 \begin{array}{c}
 \begin{bmatrix} 1 & 2 & 1 & 1 \\ 2 & 0 & 1 & 3 \\ 1 & 1 & 0 & 2 \\ 1 & 3 & 2 & 2 \\ 0 & 2 & 0 & 1 \\ 2 & 1 & 1 & 2 \\ 0 & 1 & 1 & 1 \\ 1 & 2 & 1 & 0 \\ 1 & 2 & 0 & 1 \\ 2 & 1 & 1 & 3 \\ 0 & 1 & 3 & 3 \\ 1 & 3 & 3 & 2 \end{bmatrix} \\
 \text{Input Features } X \text{ (unrolled)}
 \end{array}
 =
 \begin{array}{c}
 \begin{bmatrix} 12 & 18 & 13 & 22 \\ 10 & 20 & 15 & 22 \end{bmatrix} \\
 \text{Output Features } Y
 \end{array}$$

Example: AlexNet (2012)

- AlexNet wins the **ImageNet classification competition** with ~10% points higher accuracy than state-of-the-art
 - Krizhevsky et al., “**ImageNet Classification with Deep Convolutional Neural Networks**”, NIPS 2012.

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called “dropout” that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.

Example: GoogLeNet (2014)

- Google improves accuracy by **adding more network layers**
 - From 8 in AlexNet to 22 in GoogLeNet
 - Szegedy et al., “**Going Deeper with Convolutions**”, CVPR 2015.

Going Deeper with Convolutions

Christian Szegedy¹, Wei Liu², Yangqing Jia¹, Pierre Sermanet¹, Scott Reed³,
Dragomir Anguelov¹, Dumitru Erhan¹, Vincent Vanhoucke¹, Andrew Rabinovich⁴

¹Google Inc. ²University of North Carolina, Chapel Hill

³University of Michigan, Ann Arbor ⁴Magic Leap Inc.

¹{szegedy, jiaayq, sermanet, dragomir, dimitru, vanhoucke}@google.com

²wliu@cs.unc.edu, ³reedscott@umich.edu, ⁴arabinovich@magicleap.com

Example: ResNet (2015)

- He et al., "Deep Residual Learning for Image Recognition", CVPR 2016.

Deep Residual Learning for Image Recognition

Kaiming He

Xiangyu Zhang

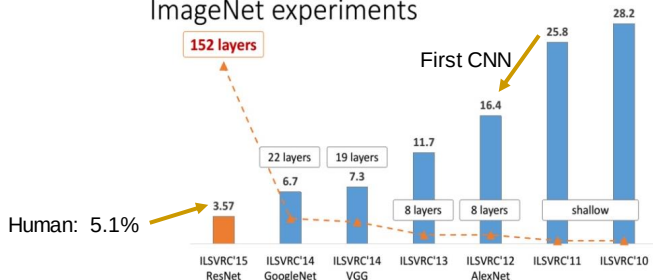
Shaoqing Ren

Jian Sun

Microsoft Research

{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

ImageNet experiments



ImageNet Classification top-5 error (%)

Systolic Computation Example: Convolution (I)

■ Convolution

- Used in filtering, pattern matching, correlation, polynomial evaluation, etc ...
- Many **image processing** tasks
- **Machine learning**: up to hundreds of **convolutional layers** in Convolutional Neural Networks (CNN)

Given the sequence of weights $\{w_1, w_2, \dots, w_k\}$
and the input sequence $\{x_1, x_2, \dots, x_n\}$,
compute the result sequence $\{y_1, y_2, \dots, y_{n+1-k}\}$
defined by

$$y_i = w_1x_i + w_2x_{i+1} + \dots + w_kx_{i+k-1}$$

Systolic Computation Example: Convolution (II)

- $y_1 = w_1x_1 + w_2x_2 + w_3x_3$
- $y_2 = w_1x_2 + w_2x_3 + w_3x_4$
- $y_3 = w_1x_3 + w_2x_4 + w_3x_5$

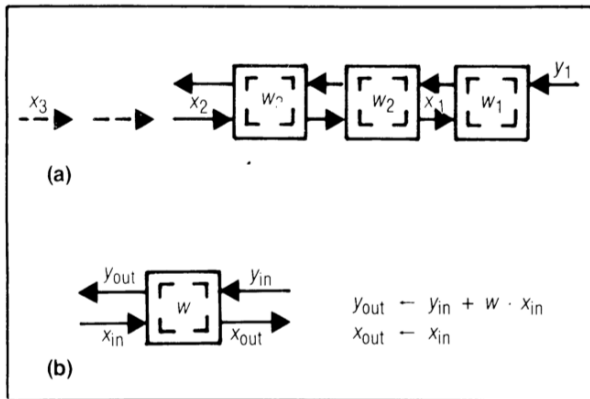


Figure 8. Design W1: systolic convolution array (a) and cell (b) where w_i 's stay and x_i 's and y_i 's move systolically in opposite directions.

Systolic Computation Example: Convolution (III)

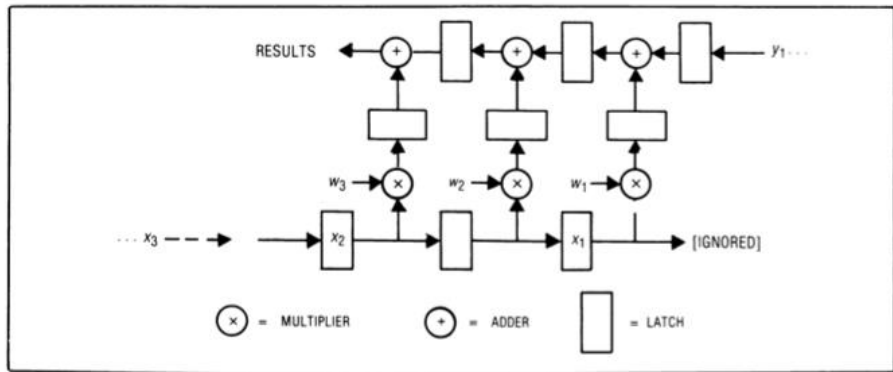


Figure 10. Overlapping the executions of multiply and add in design W1.

- Worthwhile to implement adder and multiplier separately to allow overlapping of add/mul executions

Systolic Computation Example: Convolution (IV)

- One needs to **carefully orchestrate** when **data elements are input to the array**
- And when **output is buffered**
- This gets more involved when
 - Array dimensionality increases
 - PEs are less predictable in terms of latency

Example 2D Systolic Array Computation

- Multiply two 3x3 matrices (inputs)
 - Keep the final result in PE accumulators

$$\begin{bmatrix} c_{00} & c_{01} & c_{02} \\ c_{10} & c_{11} & c_{12} \\ c_{20} & c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \end{bmatrix}$$

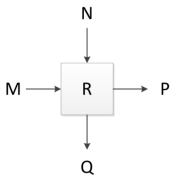
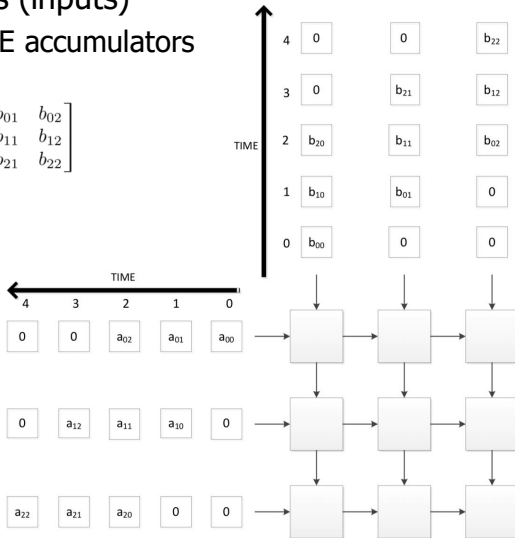


Figure 1: A systolic array processing element

$$\begin{aligned} P &= M \\ Q &= N \\ R &= R + M \cdot N \end{aligned}$$



Two-Dimensional Systolic Arrays

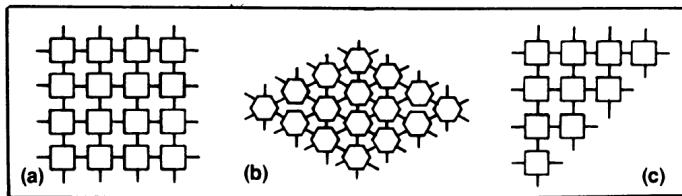


Figure 11. Two-dimensional systolic arrays: (a) type R, (b) type H, and (c) type T.

To a given problem there could be both one- and two-dimensional systolic array solutions. For example, two-dimensional convolution can be performed by a one-dimensional systolic array^{24,25} or a two-dimensional systolic array.⁶ When the memory speed is more than cell speed, two-dimensional systolic arrays such as those depicted in Figure 11 should be used. At each cell cycle, all the I/O ports on the array boundaries can input or output data items to or from the memory; as a result, the available memory bandwidth can be fully utilized. Thus, the choice of a one- or two-dimensional scheme is very dependent on how cells and memories will be implemented.

Combinations

- Systolic arrays can be chained together to form powerful systems
- This systolic array is capable of producing on-the-fly least-squares fit to all the data that has arrived up to any given moment

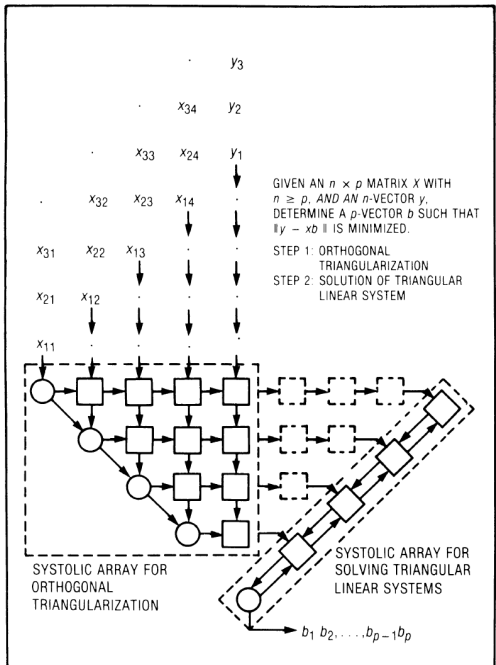


Figure 12. On-the-fly least-squares solutions using one- and two-dimensional systolic arrays, with $p = 4$.

Systolic Arrays: Pros and Cons

■ Advantages:

- ❑ **Principled**: Efficiently makes use of limited memory bandwidth, balances computation to I/O bandwidth availability
- ❑ **Specialized** (computation needs to fit PE organization/functions)
 - improved **efficiency, simple design, high concurrency/performance**
 - good to do **more with less memory bandwidth** requirement

■ Downside:

- ❑ **Specialized**
 - **not generally applicable** because computation needs to fit the PE functions/organization

More Programmability in Systolic Arrays

- Each PE in a systolic array
 - Can store multiple “weights”
 - Weights can be selected on the fly
 - Eases implementation of, e.g., adaptive filtering
- Taken further
 - Each PE can have its own data and instruction memory
 - Data memory → to store partial/temporary results, constants
 - Leads to **stream processing, pipeline parallelism**
 - More generally, **staged execution**

Pipeline-Parallel (Pipelined) Programs

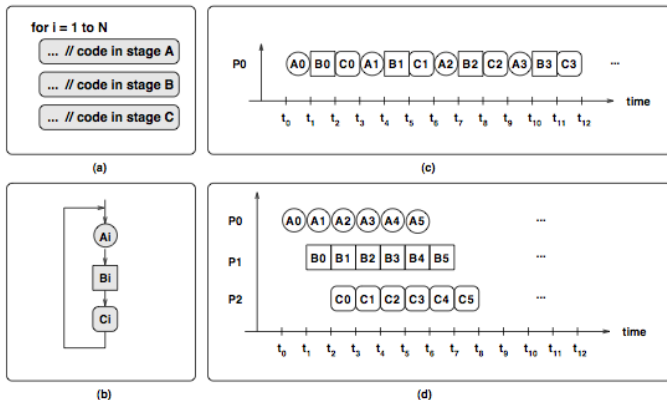
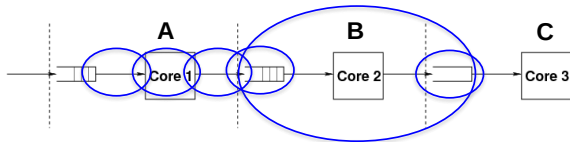


Figure 1. (a) The code of a loop, (b) Each iteration is split into 3 pipeline stages: A, B, and C. Iteration i comprises A_i , B_i , C_i . (c) Sequential execution of 4 iterations. (d) Parallel execution of 6 iterations using pipeline parallelism on a three-core machine. Each stage executes on one core.

Stages of Pipelined Programs

- Loop iterations are divided into **code segments called stages**
- Threads execute stages on different cores

```
loop {  
  Compute1 A  
  Compute2 B  
  Compute3 C  
}
```



Pipelined File Compression Example

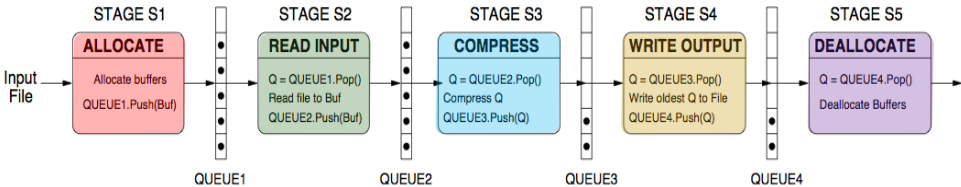


Figure 3. File compression algorithm executed using pipeline parallelism

Systolic Array: Advantages & Disadvantages

■ Advantages

- ❑ Makes **multiple uses of each data item** → reduced need for fetching/refetching → better use of memory bandwidth
- ❑ **High concurrency**
- ❑ Regular design (both data and control flow)

■ Disadvantages

- ❑ **Not good at exploiting irregular parallelism**
- ❑ Relatively special purpose → need software, programmer support to be a general purpose model

Example Systolic Array: The WARP Computer

- HT Kung, CMU, 1984-1988
- Linear array of 10 cells, each cell a 10 Mflop programmable processor
- Attached to a general purpose host machine
- HLL and optimizing compiler to program the systolic array
- Used extensively to accelerate vision and robotics tasks
- Annaratone et al., "Warp Architecture and Implementation," ISCA 1986.
- Annaratone et al., "The Warp Computer: Architecture, Implementation, and Performance," IEEE TC 1987.

The WARP Computer

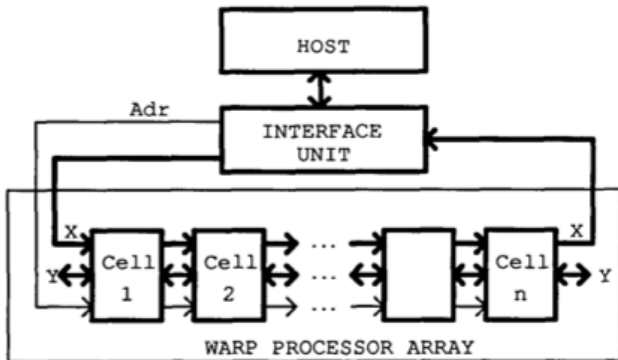


Figure 1: Warp system overview

The WARP Cell

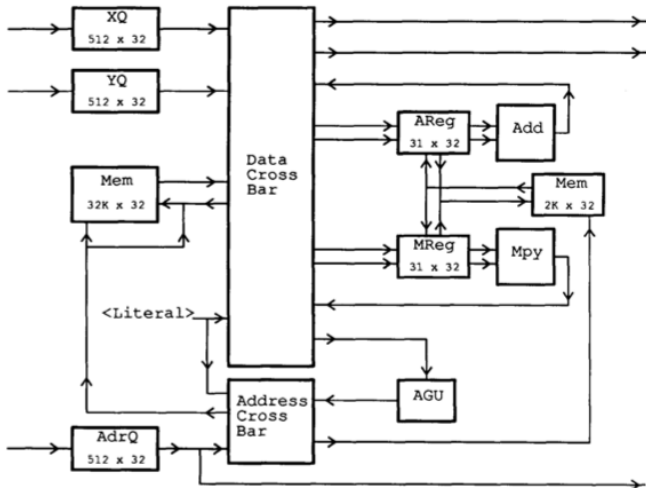


Figure 2: Warp cell data path

An Example Modern Systolic Array: TPU (I)

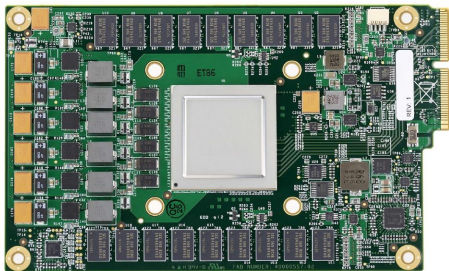


Figure 3. TPU Printed Circuit Board. It can be inserted in the slot for an SATA disk in a server, but the card uses PCIe Gen3 x16.

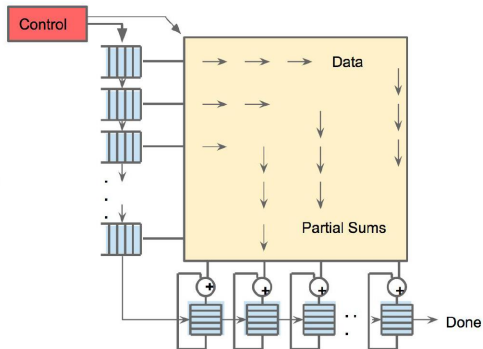
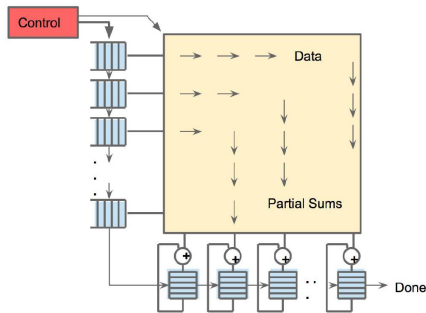


Figure 4. Systolic data flow of the Matrix Multiply Unit. Software has the illusion that each 256B input is read at once, and they instantly update one location of each of 256 accumulator RAMs.

Jouppi et al., “[In-Datcenter Performance Analysis of a Tensor Processing Unit](#)”, ISCA 2017.

An Example Modern Systolic Array: TPU (II)

As reading a large SRAM uses much more power than arithmetic, the matrix unit uses systolic execution to save energy by reducing reads and writes of the Unified Buffer [Kun80][Ram91][Ovt15b]. Figure 4 shows that data flows in from the left, and the weights are loaded from the top. A given 256-element multiply-accumulate operation moves through the matrix as a diagonal wavefront. The weights are preloaded, and take effect with the advancing wave alongside the first data of a new block. Control and data are pipelined to give the illusion that the 256 inputs are read at once, and that they instantly update one location of each of 256 accumulators. From a correctness perspective, software is unaware of the systolic nature of the matrix unit, but for performance, it does worry about the latency of the unit.



Jouppi et al., "In-Datacenter Performance Analysis of a Tensor Processing Unit", ISCA 2017.

Recall: Example 2D Systolic Array Computation

- Multiply two 3x3 matrices (inputs)
 - Keep the final result in PE accumulators

$$\begin{bmatrix} c_{00} & c_{01} & c_{02} \\ c_{10} & c_{11} & c_{12} \\ c_{20} & c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \end{bmatrix}$$

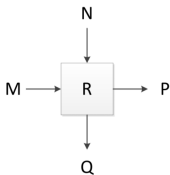
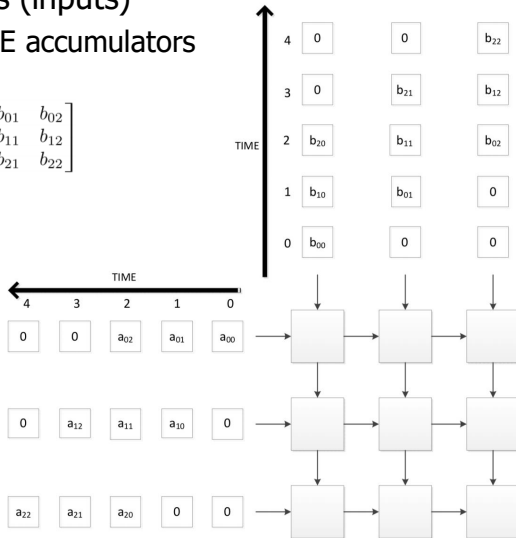


Figure 1: A systolic array processing element

$$\begin{aligned} P &= M \\ Q &= N \\ R &= R + M \cdot N \end{aligned}$$



An Example Modern Systolic Array: TPU2



<https://www.nextplatform.com/2017/05/17/first-depth-look-googles-new-second-generation-tpu/>

4 TPU chips
vs 1 chip in TPU1

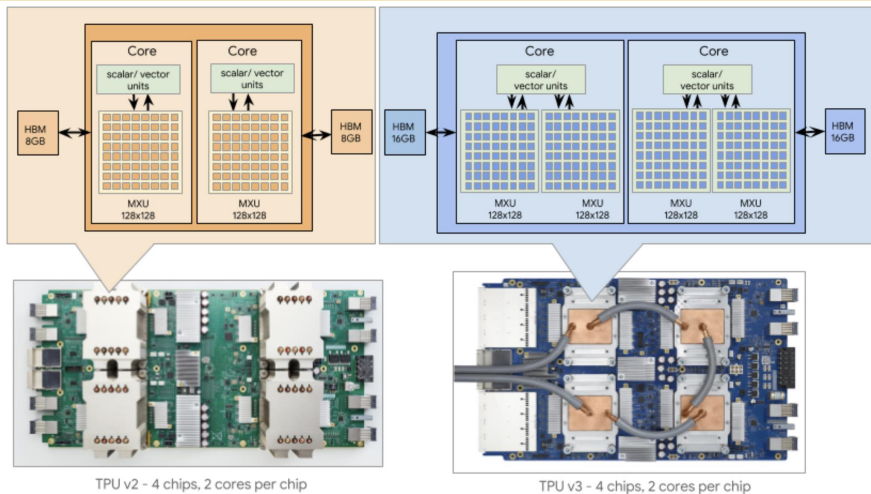
High Bandwidth Memory
vs DDR3

Floating point operations
vs FP16

45 TFLOPS per chip
vs 23 TOPS

Designed for training
and inference
vs only inference

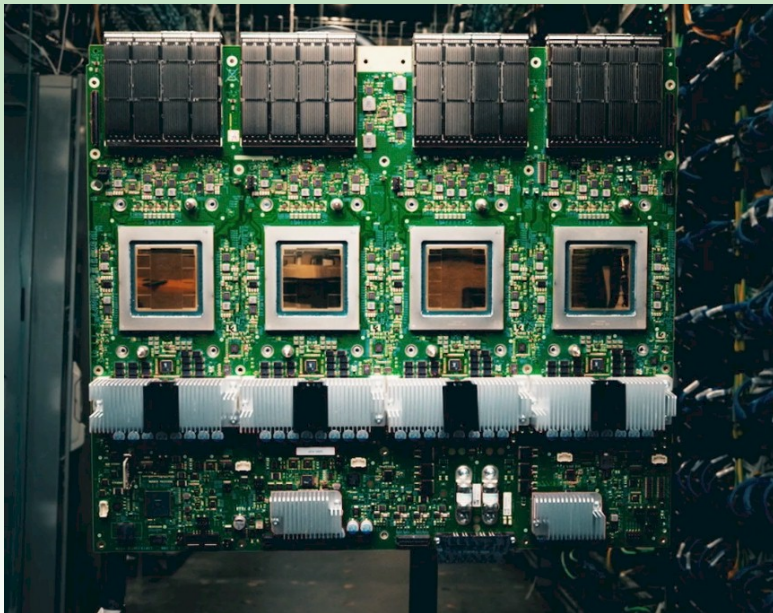
An Example Modern Systolic Array: TPU3



32GB HBM per chip
vs 16GB HBM in TPU2

4 Matrix Units per chip
vs 2 Matrix Units in TPU2

90 TFLOPS per chip
vs 45 TFLOPS in TPU2



Źródło: <https://www.nextplatform.com/2025/04/09/with-ironwood-tpu-google-pushes-the-ai-accelerator-to-the-floor/>



TPU v4



TPU v5p



Ironwood

	2022	2023	2025
Pod Size (chips)	4896	8960	9216
HBM Bandwidth/ Capacity	32 GB @ 1.2 TBps HBM	95 GB @ 2.8 TBps HBM	192 GB @ 7.4 TBps HBM
Peak Flops per chip	275 TFLOPS	459 TFLOPS	4614 TFLOPS

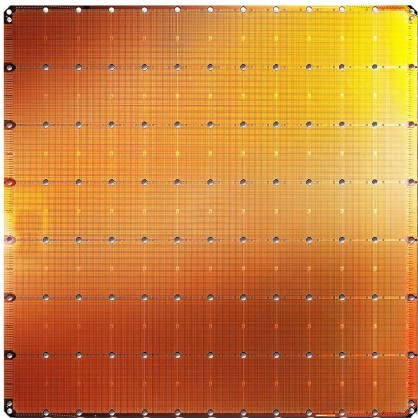
Źródło: <https://blog.google/products/google-cloud/ironwood-tpu-age-of-inference/>

Peak Flops Per Watt (TDP)



Źródło: <https://blog.google/products/google-cloud/ironwood-tpu-age-of-inference/>

Cerebras's Wafer Scale Engine (2019)



Cerebras WSE

1.2 Trillion transistors
46,225 mm²

- The largest ML accelerator chip
- 400,000 cores



Largest GPU

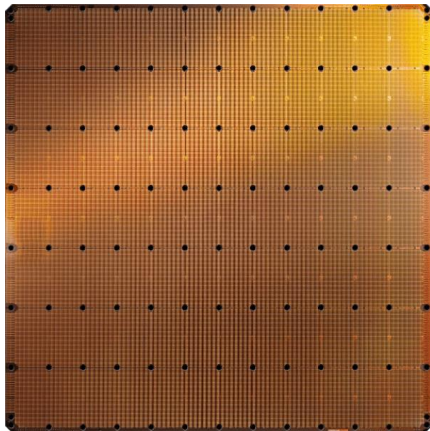
21.1 Billion transistors
815 mm²

NVIDIA TITAN V

<https://www.anandtech.com/show/14758/hot-chips-31-live-blogs-cerebras-wafer-scale-deep-learning>

<https://www.cerebras.net/cerebras-wafer-scale-engine-why-we-need-big-chips-for-deep-learning>

Cerebras's Wafer Scale Engine-2 (2021)



Cerebras WSE-2
2.6 Trillion transistors
46,225 mm²

- The largest ML accelerator chip
- 850,000 cores



Largest GPU
54.2 Billion transistors
826 mm²

NVIDIA Ampere GA100

<https://www.anandtech.com/show/14758/hot-chips-31-live-blogs-cerebras-wafer-scale-deep-learning>

<https://www.cerebras.net/cerebras-wafer-scale-engine-why-we-need-big-chips-for-deep-learning>

Approaches to (Instruction-Level) Concurrency

- Pipelining
- Fine-Grained Multithreading
- Out-of-order Execution
- Dataflow (at the ISA level)
- Superscalar Execution
- VLIW
- Systolic Arrays
- Decoupled Access Execute
- SIMD Processing (Vector and Array processors, GPUs)

Flynn's Taxonomy of Computers

- Mike Flynn, “**Very High-Speed Computing Systems**,” Proc. of IEEE, 1966
- **SISD**: Single instruction operates on single data element
- **SIMD**: Single instruction operates on multiple data elements
 - Array processor
 - Vector processor
- **MISD**: Multiple instructions operate on single data element
 - Closest form: systolic array processor, streaming processor
- **MIMD**: Multiple instructions operate on multiple data elements (multiple instruction streams)
 - Multiprocessor
 - Multithreaded processor

Flynn's Taxonomy of Computers

- Mike Flynn, “**Very High-Speed Computing Systems**,” Proc. of IEEE, 1966

Very High-Speed Computing Systems

MICHAEL J. FLYNN, MEMBER, IEEE

Abstract—Very high-speed computers may be classified as follows:

- 1) Single Instruction Stream—Single Data Stream (SISD)
- 2) Single Instruction Stream—Multiple Data Stream (SIMD)
- 3) Multiple Instruction Stream—Single Data Stream (MISD)
- 4) Multiple Instruction Stream—Multiple Data Stream (MIMD).

“Stream,” as used here, refers to the sequence of data or instructions as seen by the machine during the execution of a program.

The constituents of a system: storage, execution, and instruction handling (branching) are discussed with regard to recent developments and/or systems limitations. The constituents are discussed in terms of concurrent SISD

Manuscript received June 30, 1966; revised August 16, 1966. This work was performed under the auspices of the U. S. Atomic Energy Commission.

The author is with Northwestern University, Evanston, Ill., and Argonne National Laboratory, Argonne, Ill.

systems (CDC 6600 series and, in particular, IBM Model 90 series), since multiple stream organizations usually do not require any more elaborate components.

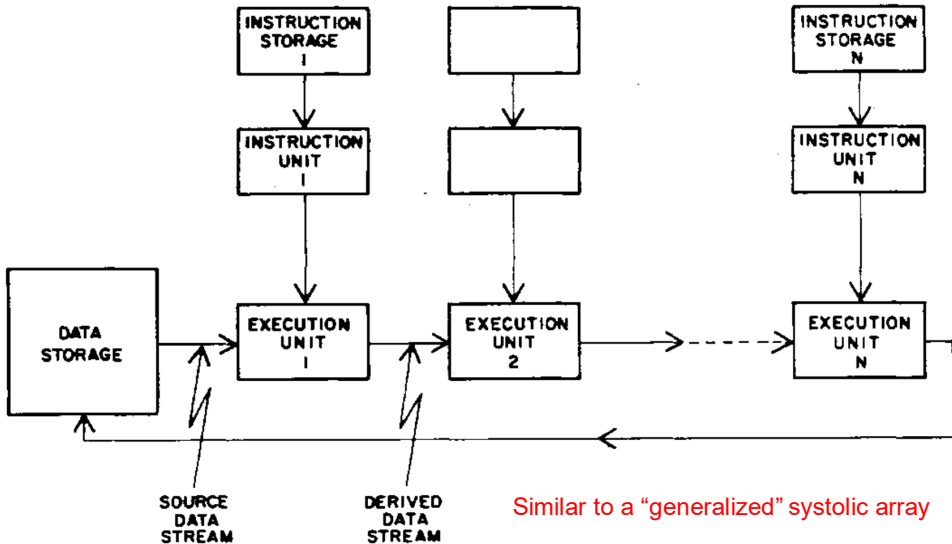
Representative organizations are selected from each class and the arrangement of the constituents is shown.

INTRODUCTION

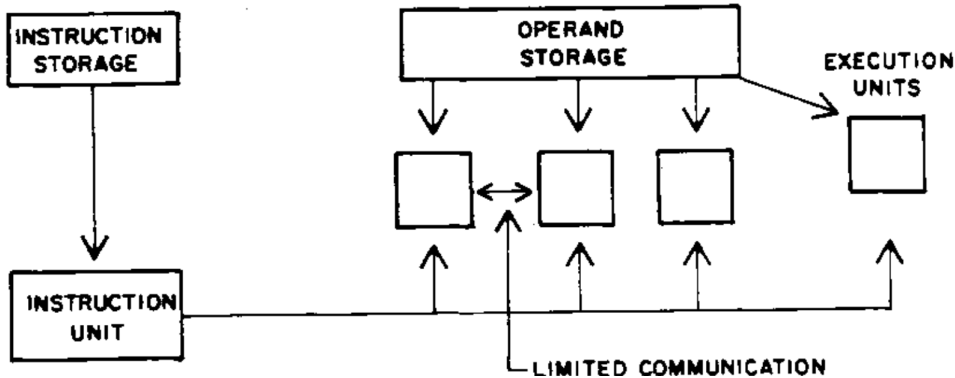
MANY SIGNIFICANT scientific problems require the use of prodigious amounts of computing time.

In order to handle these problems adequately, the large-scale scientific computer has been developed. This computer addresses itself to a class of problems characterized by having a high ratio of computing requirement to input/output requirements (a partially de facto situation

MISD Example from Flynn



SIMD Example from Flynn



Similar to an “array processor”

Flynn's Taxonomy of Computers

- Mike Flynn, “**Very High-Speed Computing Systems**,” Proc. of IEEE, 1966
- **SISD**: Single instruction operates on single data element
- **SIMD**: Single instruction operates on multiple data elements
 - Array processor
 - Vector processor
- **MISD**: Multiple instructions operate on single data element
 - Closest form: systolic array processor, streaming processor
- **MIMD**: Multiple instructions operate on multiple data elements (multiple instruction streams)
 - Multiprocessor
 - Multithreaded processor

Data Parallelism

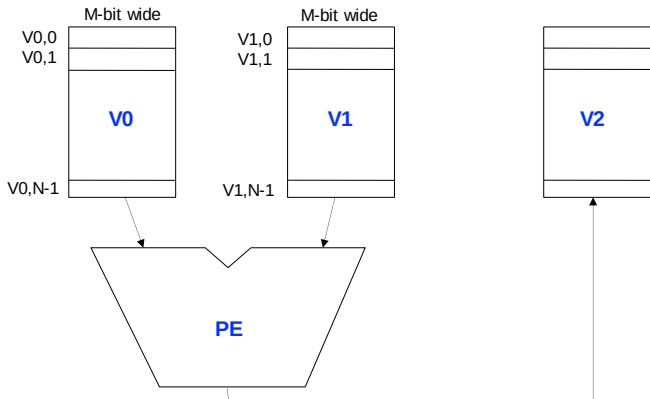
- Concurrency arises from performing the **same operation on different pieces of data**
 - Single instruction multiple data (SIMD)
 - E.g., dot product of two vectors
- Contrast with data flow
 - Concurrency arises from executing different operations in parallel (in a data driven manner)
- Contrast with thread (“control”) parallelism
 - Concurrency arises from executing different threads of control in parallel
- SIMD exploits operation-level parallelism on different data
 - Same operation concurrently applied to different pieces of data
 - A form of ILP where instruction happens to be the same across data

SIMD Processing

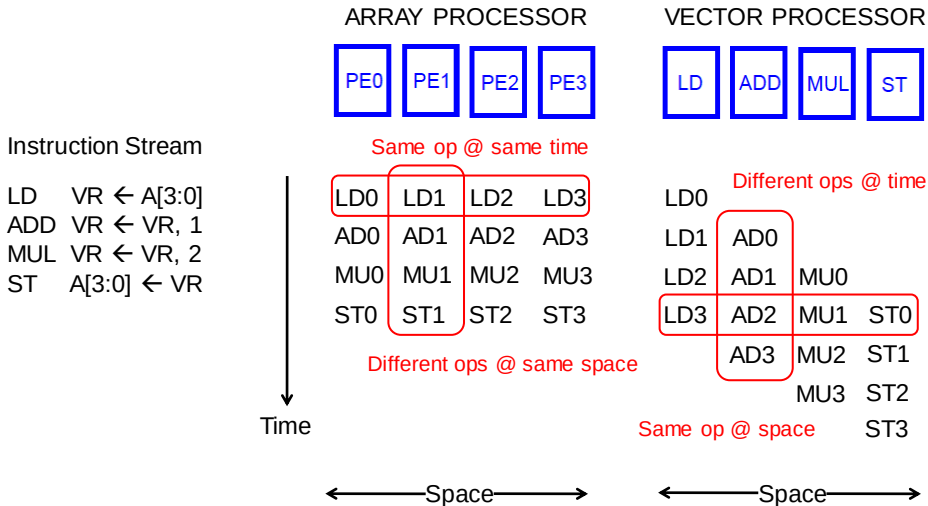
- Single instruction operates on multiple data elements
 - In time or in space
- Multiple processing elements (PEs), i.e., execution units
- Time-space duality
 - **Array processor**: Instruction operates on multiple data elements at the **same time** using **different spaces (PEs)**
 - **Vector processor**: Instruction operates on multiple data elements in **consecutive time steps** using the **same space (PE)**

Storing Multiple Data Elements: Vector Registers

- Each **vector data register** holds N M-bit values
 - Each register stores a vector
 - Not a (single) scalar value as we saw before

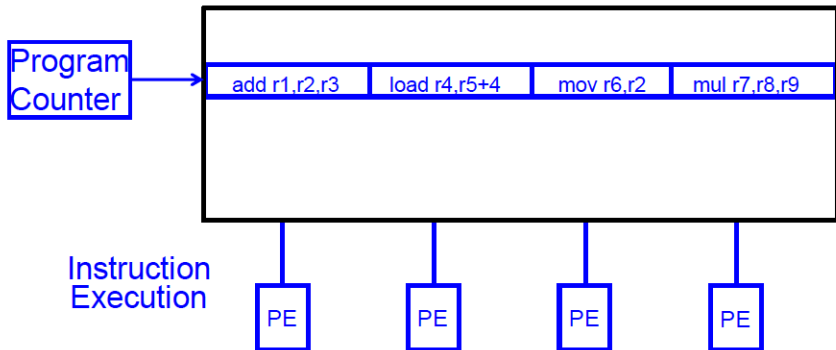


Array vs. Vector Processors



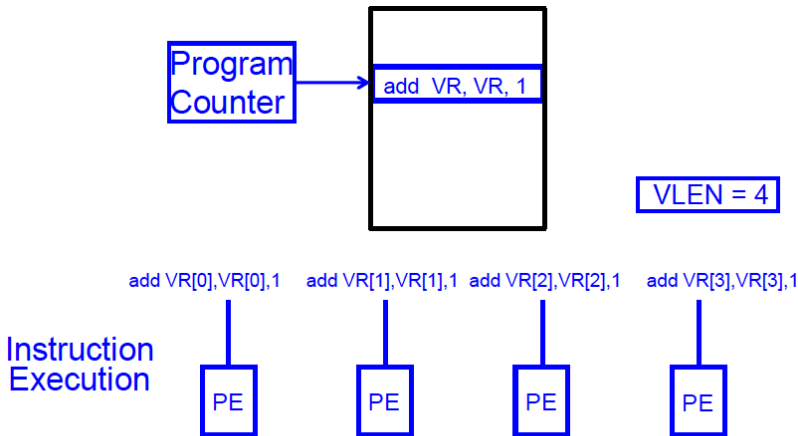
SIMD Array Processing vs. VLIW

- VLIW: **Multiple** independent **operations** packed together into a "long inst."



SIMD Array Processing vs. VLIW

- Array processor: **Single operation** on multiple (different) data elements



Vector Processors (I)

- A vector is a one-dimensional array of numbers
- Many scientific/commercial programs use vectors
 - for (i = 0; i<=49; i++)
 $C[i] = (A[i] + B[i]) / 2$
- A vector processor is one whose instructions operate on vectors rather than scalar (single data) values
- Basic requirements
 - Need to load/store vectors → vector registers (contain vectors)
 - Need to operate on vectors of different lengths → vector length register (VLEN)
 - Elements of a vector might be stored apart from each other in memory → vector stride register (VSTR)
 - Stride: distance in memory between two elements of a vector

Vector Stride Example: Matrix Multiply

- A and B matrices, both stored in memory in **row-major order**

A_0	0	1	2	3	4	5
	6	7	8	9	10	11

$$A_{4 \times 6} B_{6 \times 10} \rightarrow C_{4 \times 10}$$

Dot product of each row vector of A with each column vector of B

B ₀	0	1	2	3	4	5	6	7	8	9
	10	11	12	13	14	15	16	17	18	19
	20									
	30									
	40									
	50									

Linear Memory

A	0
	1
	2
	3
	4
	5
	6
B	0
	1
	2
	3
	4
	5
	6
	7
	8
	9
	10

- Load A's row 0 (A_{00} through A_{05}) into vector register V_1
 - Each time, increment address by 1 to access the next column
 - Accesses have a stride of 1
- Load B's column 0 (B_{00} through B_{50}) into vector register V_2
 - Each time, increment address by 10 to access the next row
 - Accesses have a stride of 10

Vector Processors (II)

- A vector instruction performs an operation on each element in consecutive cycles
 - Vector functional units are pipelined
 - Each pipeline stage operates on a different data element
- Vector instructions allow deeper pipelines
 - No intra-vector dependencies → no hardware interlocking needed within a vector
 - No control flow within a vector
 - Known stride allows easy address calculation for all vector elements
 - Enables easy loading (or even early loading, i.e., prefetching) of vectors into registers/cache/memory

Vector Processor Advantages

+ No dependencies within a vector

- ❑ Pipelining & parallelization work really well
- ❑ Can have very deep pipelines (without the penalty of deep pipelines)

+ Each instruction generates a lot of work (i.e., operations)

- ❑ Reduces instruction fetch bandwidth requirements
- ❑ Amortizes instruction fetch and control overhead over many data
--> Leads to high energy efficiency per operation

+ No need to explicitly code loops

- ❑ Fewer branches in the instruction sequence

+ Highly regular memory access pattern

Vector Processor Disadvantages

- Works (only) if parallelism is regular (data/SIMD parallelism)
 - ++ Vector operations
- Very inefficient if parallelism is irregular
 - How about searching for a key in a linked list?

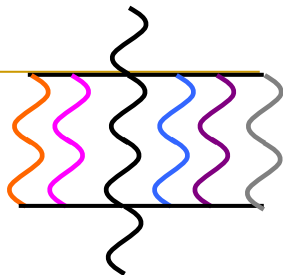
To program a vector machine, the compiler or hand coder must make the data structures in the code fit nearly exactly the regular structure built into the hardware. That's hard to do in first place, and just as hard to change. One tweak, and the low-level code has to be rewritten by a very smart and dedicated programmer who knows the hardware and often the subtleties of the application area. Often the rewriting is

Amdahl's Law

■ Amdahl's Law

- f: Parallelizable fraction of a program
- N: Number of processors

$$\text{Speedup} = \frac{1}{1 - f + \frac{f}{N}}$$



- Amdahl, “[Validity of the single processor approach to achieving large scale computing capabilities](#),” AFIPS 1967.

- **Maximum speedup limited by serial portion: Serial bottleneck**
- All parallel machines “suffer from” the serial bottleneck

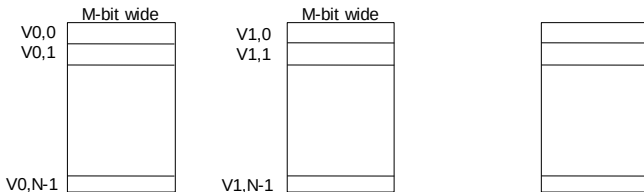
Vector Processor Limitations

- Memory (bandwidth) can easily become a bottleneck, especially if
 1. compute/memory operation balance is not maintained
 2. data is not mapped appropriately to memory banks

Vector Processing in More Depth

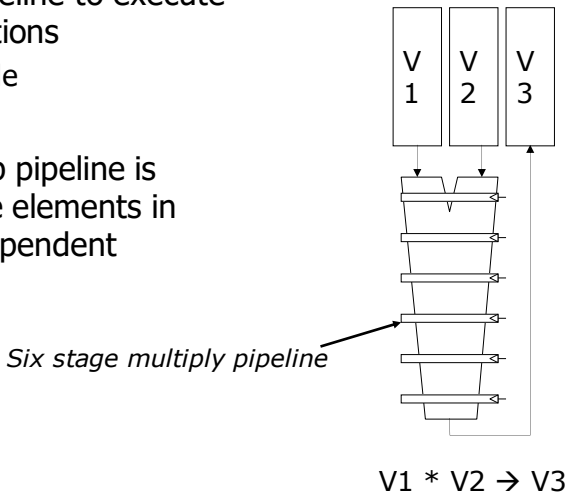
Vector Registers

- Each **vector data register** holds N M-bit values
- **Vector control registers**: VLEN, VSTR, VMASK
- Maximum VLEN can be N
 - Maximum number of elements stored in a vector register
- **Vector Mask Register (VMASK)**
 - Indicates which elements of vector to operate on
 - Set by vector test instructions
 - e.g., $VMASK[i] = (V_k[i] == 0)$



Vector Functional Units

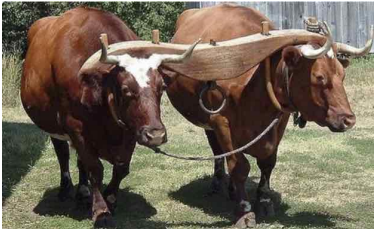
- Use a deep pipeline to execute element operations
→ fast clock cycle
- Control of deep pipeline is simple because elements in vector are independent



Seymour Cray, Leader in Supercomputer Design



"If you were plowing a field, which would you rather use: **Two strong oxen** or **1024 chickens**?"

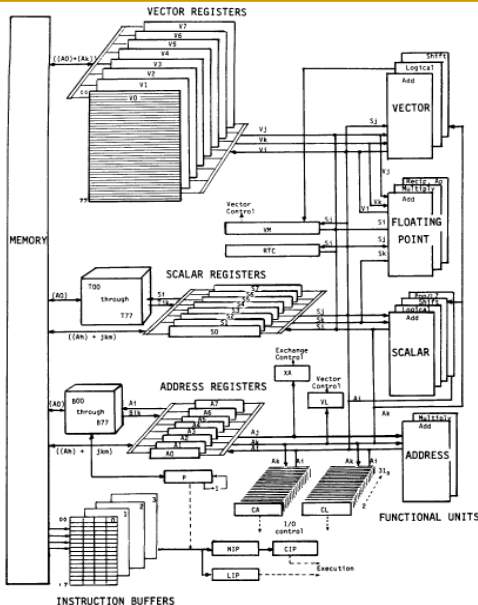


© amityrebecca / Pinterest. <https://www.pinterest.ch/pin/473018767088408061/>



© Scott Sinkler / Corbis. <http://america.aljazeera.com/articles/2015/2/20/the-short-brutal-life-of-male-chickens.html>

Vector Machine Organization (CRAY-1)



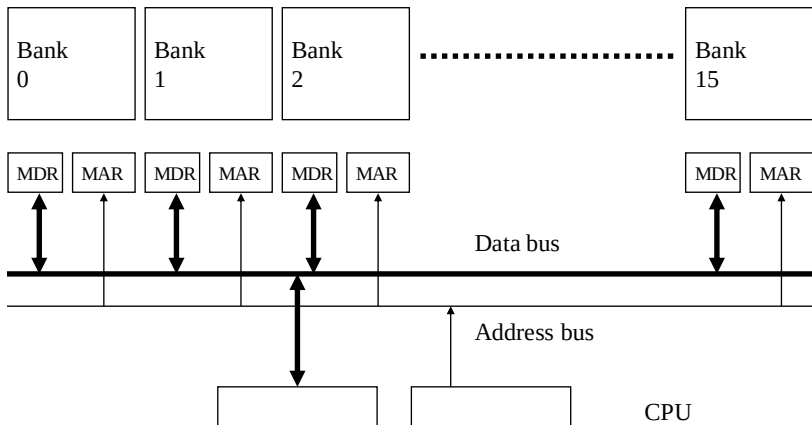
- CRAY-1
- Russell, "The CRAY-1 computer system," CACM 1978.
- Scalar and vector modes
- 8 64-element vector registers
- 64 bits per element
- 16 memory banks
- 8 64-bit scalar registers
- 8 24-bit address registers

Loading/Storing Vectors from/to Memory

- Requires loading/storing multiple elements
- Elements separated from each other by a constant distance (stride)
 - Assume stride = 1 for now
- Elements can be loaded in consecutive cycles if we can start the load of one element per cycle
 - Can sustain a throughput of one element per cycle
- Question: How do we achieve this with a memory that takes **more than 1 cycle to access**?
- Answer: **Bank** the memory; **interleave** the elements across banks

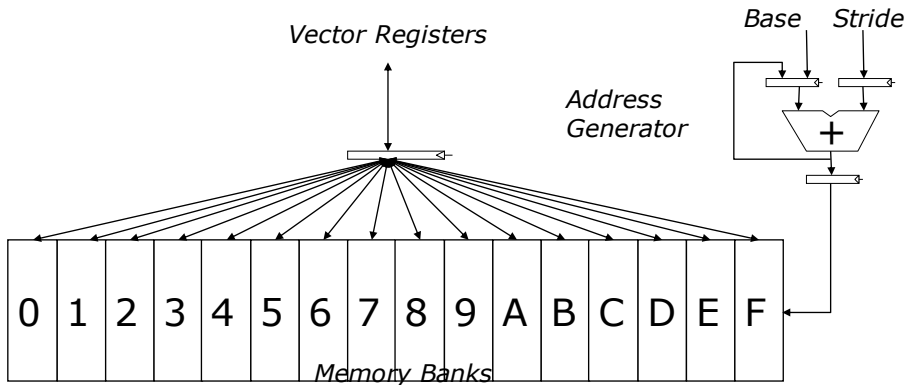
Memory Banking

- Memory is divided into **banks** that can be accessed independently; banks share address and data buses (to minimize pin cost)
- Can start and complete one bank access per cycle
- Can sustain N concurrent accesses if all N go to different banks



Vector Memory System

- Next address = Previous address + Stride
- If (stride == 1) && (consecutive elements interleaved across banks) && (number of banks >= bank latency), then
 - we can sustain 1 element/cycle throughput



Scalar Code Example: Element-Wise Avg.

- For I = 0 to 49
 - ▣ $C[i] = (A[i] + B[i]) / 2$
- Scalar code (instruction and its latency)

MOVI R0 = 50	1	
MOVA R1 = A	1	304 dynamic instructions
MOVA R2 = B	1	
MOVA R3 = C	1	
X: LD R4 = MEM[R1++]	11	;autoincrement addressing
LD R5 = MEM[R2++]	11	
ADD R6 = R4 + R5	4	
SHFR R7 = R6 >> 1	1	
ST MEM[R3++] = R7	11	
DECBNZ R0, X	2	;decrement and branch if NZ

Scalar Code Execution Time (In Order)

- Scalar execution time on an in-order processor with 1 bank
 - First two loads in the loop cannot be pipelined: 2×11 cycles
 - $4 + 50 \times 40 = 2004$ cycles
- Scalar execution time on an in-order processor with 16 banks (word-interleaved: consecutive words are stored in consecutive banks)
 - First two loads in the loop can be pipelined
 - $4 + 50 \times 30 = 1504$ cycles
- Why 16 banks?
 - 11-cycle memory access latency
 - Having 16 (>11) banks ensures there are enough banks to overlap enough memory operations to cover memory latency

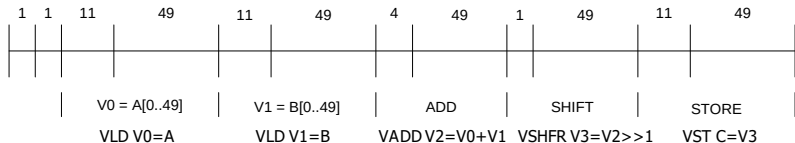
Vectorizable Loops

- A loop is **vectorizable** if each iteration is independent of any other
- For $I = 0$ to 49
 - $C[i] = (A[i] + B[i]) / 2$
- Vectorized loop (each instruction and its latency):

MOVI VLEN = 50	1	7 dynamic instructions
MOVI VSTR = 1	1	
VLD V0 = A	$11 + \text{VLEN} - 1$	
VLD V1 = B	$11 + \text{VLEN} - 1$	
VADD V2 = V0 + V1	$4 + \text{VLEN} - 1$	
VSHFR V3 = V2 >> 1	$1 + \text{VLEN} - 1$	
VST C = V3	$11 + \text{VLEN} - 1$	

Basic Vector Code Performance

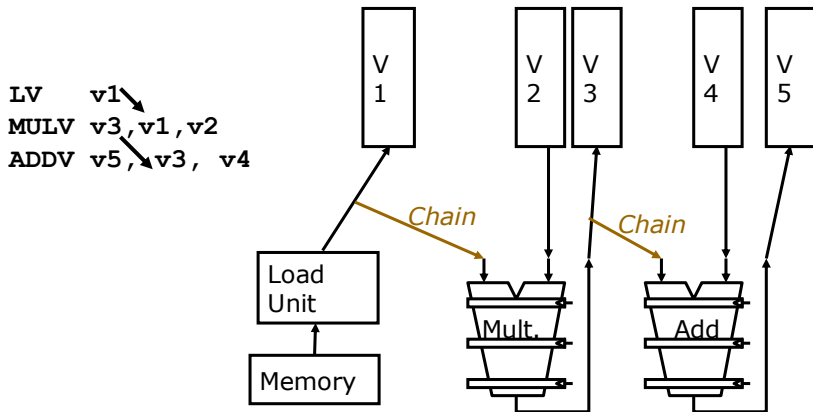
- Assume **no chaining** (no vector data forwarding)
 - i.e., output of a vector functional unit cannot be used as the direct input of another
 - **The entire vector register needs to be ready** before any element of it can be used as part of another operation
- One memory port (one address generator)
- 16 memory banks (word-interleaved)



- 285 cycles

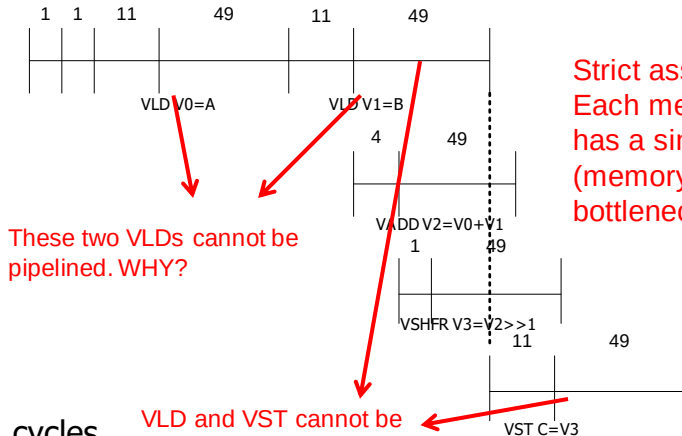
Vector Chaining

- **Vector chaining**: Data forwarding from one vector functional unit to another



Vector Code Performance - Chaining

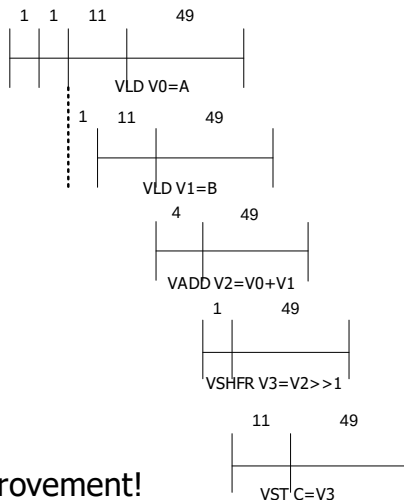
- **Vector chaining:** Data forwarding from one vector functional unit to another



- 182 cycles

Vector Code Performance – Multiple Memory Ports

- Chaining and 2 load ports, 1 store port in each bank



- 79 cycles
- 19X perf. improvement!

Questions (I)

- What if # data elements > # elements in a vector register?
 - Idea: Break loops so that each iteration operates on # elements in a vector register
 - E.g., 527 data elements, 64-element VREGs
 - 8 iterations where VLEN = 64
 - 1 iteration where VLEN = 15 (need to change value of VLEN)
 - Called **vector stripmining**

(Vector) Stripmining

Surface mining, including **strip mining**, **open-pit mining** and **mountaintop removal mining**, is a broad category of **mining** in which soil and rock overlying the mineral deposit (the **overburden**) are removed, in contrast to **underground mining**, in which the overlying rock is left in place, and the mineral removed through shafts or tunnels. Surface mining began in the mid-sixteenth century^[1] and is practiced throughout the world, although the majority of surface coal mining occurs in North America.^[2] It gained



Coal strip mine in [Wyoming](#)



Questions (II)

- What if vector data is not stored in a strided fashion in memory? (**irregular memory access to a vector**)
 - Idea: Use indirection to combine/pack elements into vector registers
 - Called scatter/gather operations
 - Doing so also helps with avoiding useless computation on sparse vectors (i.e., vectors where many elements are 0)

Gather/Scatter Operations

Want to vectorize loops with indirect accesses:

```
for (i=0; i<N; i++)  
    A[i] = B[i] + C[D[i]]
```

Indexed load instruction (*Gather*)

```
LV vD, rD          # Load indices in D vector  
LVI vC, rC, vD     # Load indirect from rC base  
LV vB, rB          # Load B vector  
ADDV.D vA,vB,vC    # Do add  
SV vA, rA          # Store result
```

Gather/Scatter Operations

- Gather/scatter operations often implemented in hardware to handle **sparse vectors (matrices)** or **indirect indexing**
- Vector loads and stores use an index vector which is added to the base register to generate the addresses
- *Scatter* example

Index Vector	Data Vector (to Store)	Stored Vector (in Memory)	
0	3.14	Base+0	3.14
2	6.5	Base+1	X
6	71.2	Base+2	6.5
7	2.71	Base+3	X
		Base+4	X
		Base+5	X
		Base+6	71.2
		Base+7	2.71

Conditional Operations in a Loop

- What if some operations should not be executed on a vector (based on a dynamically-determined condition)?

```
loop:      for (i=0; i<N; i++)  
           if (a[i] != 0) then b[i]=a[i]*b[i]
```

- Idea: **Masked operations**

- VMASK register is a bit mask determining which data element should not be acted upon

VLD V0 = A

VLD V1 = B

VMASK = (V0 != 0)

VMUL V1 = V0 * V1

VST B = V1

- This is **predicated execution**. Execution is *predicated* on mask bit.

Another Example with Masking

```
for (i = 0; i < 64; ++i)
```

```
    if (a[i] >= b[i])
```

```
        c[i] = a[i]
```

```
    else
```

```
        c[i] = b[i]
```

Steps to execute the loop in SIMD code

1. Compare A, B to get
VMASK

2. Masked store of A into C

3. Complement VMASK

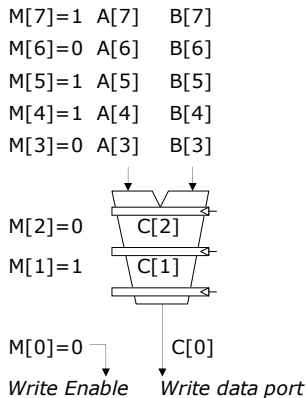
4. Masked store of B into C

A	B	VMASK
1	2	0
2	2	1
3	2	1
4	10	0
-5	-4	0
0	-3	1
6	5	1
-7	-8	1

Masked Vector Instructions

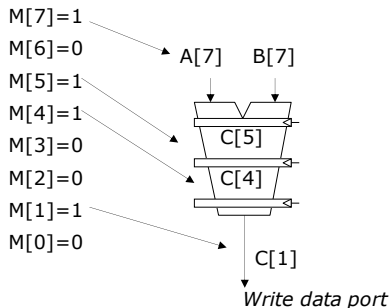
Simple Implementation

- execute all N operations, turn off result writeback according to mask



Density-Time Implementation

- scan mask vector and only execute elements with non-zero masks



Which one is better?

Tradeoffs?

Some Issues

■ Stride and banking

- As long as they are *relatively prime* to each other and there are enough banks to cover bank access latency, we can sustain 1 element/cycle throughput

■ Storage format of a matrix

- **Row major**: Consecutive elements in a row are laid out consecutively in memory
- **Column major**: Consecutive elements in a column are laid out consecutively in memory
- **You need to change the stride when accessing a row versus column**

Bank Conflicts in Matrix Multiplication

- A and B matrices, both stored in memory in **row-major order**

A_0

0	1	2	3	4	5
6	7	8	9	10	11

$$A_{4 \times 6} B_{6 \times 10} \rightarrow C_{4 \times 10}$$

Dot product of each row vector of A with each column vector of B

B_0

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20									
30									
40									
50									

- Load A's row 0 into vector register V_1

- Each time, increment address by **1** to access the next column
- Accesses have a **stride of 1**

- Load B's column 0 into vector register V_2

- Each time, increment address by 10
- Accesses have a **stride of 10**

Different strides can lead to **bank conflicts**

How do we minimize them?

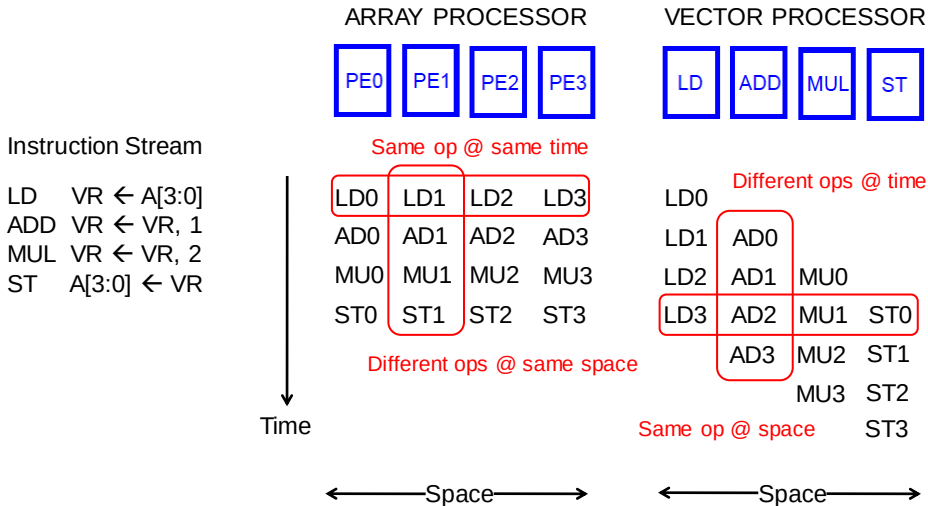
Minimizing Bank Conflicts

- More banks
- More ports in each bank
- Better data layout to match the access pattern
 - Is this always possible?
- Better mapping of address to bank
 - E.g., randomized mapping
 - Rau, "Pseudo-randomly interleaved memory," ISCA 1991.

Array vs. Vector Processors, Revisited

- Array vs. vector processor distinction is a “purist’s” distinction
- Most “modern” SIMD processors are a combination of both
 - They exploit data parallelism in both time and space
 - GPUs are a prime example we will cover in a bit more detail

Recall: Array vs. Vector Processors

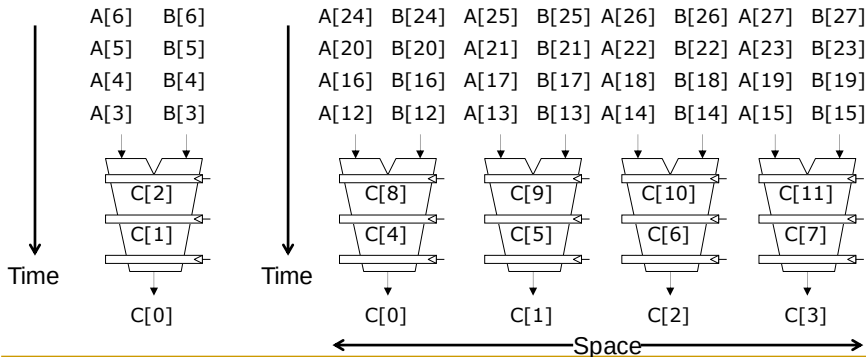


Vector Instruction Execution

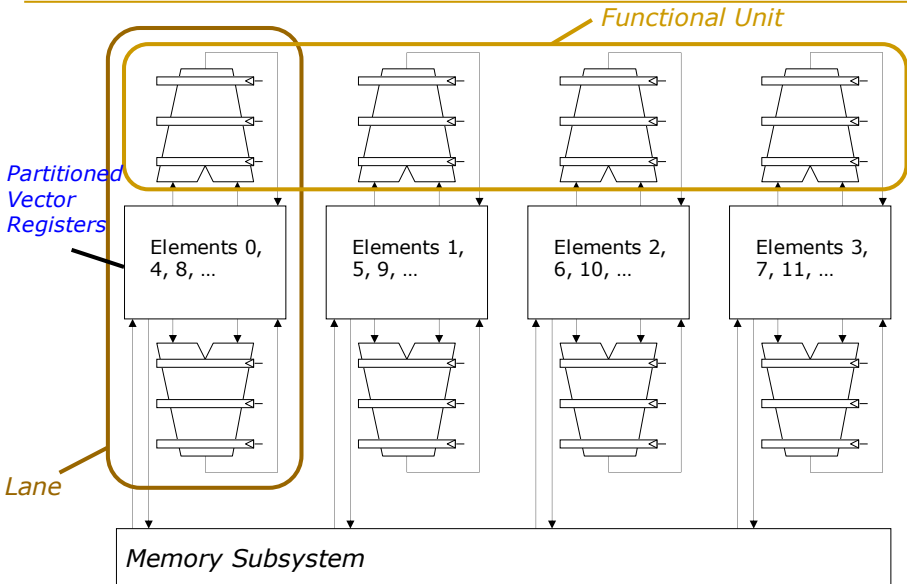
VADD A,B → C

*Execution using
one pipelined
functional unit*

*Execution using
four pipelined
functional units*



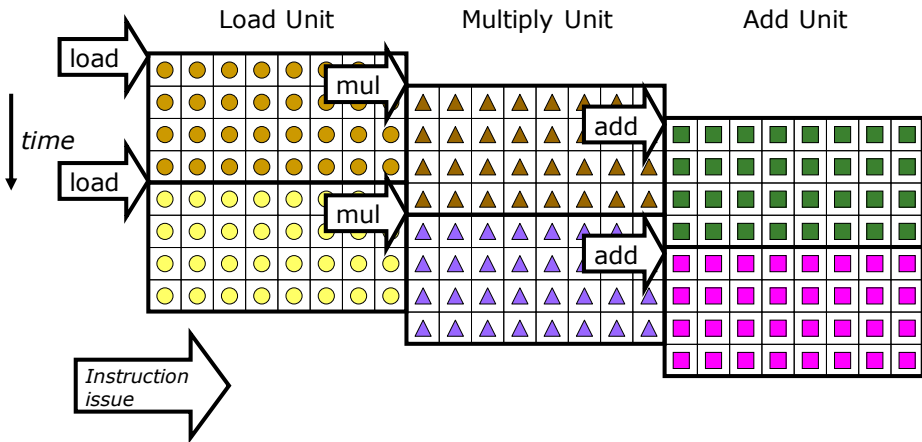
Vector Unit Structure



Vector Instruction Level Parallelism

Can overlap execution of multiple vector instructions

- Example machine has 32 elements per vector register and 8 lanes
- Completes 24 operations/cycle while issuing 1 vector instruction/cycle



Rejestry wektorowe RISC-V

- 32 rejestry wektorowe (v0-v31).
- Ponad 250 nowych instrukcji.
- CSRy: vl, vtype, vstart, vxrm, vxsat.
- Stałe zdefiniowane na etapie implementacji: VLEN, ELEN.
- Parametry: EEW, LMUL.
- Wykonanie predykatywne.
- Wsparcie dla optymalizacji out-of-order (ustawianie 1 w zamaskowanych rejestrach) (undisturbed/ignored zamaskowane/ogonowe).
- Operacje redukcji.
- Operacje zmieniające szerokość elementów.

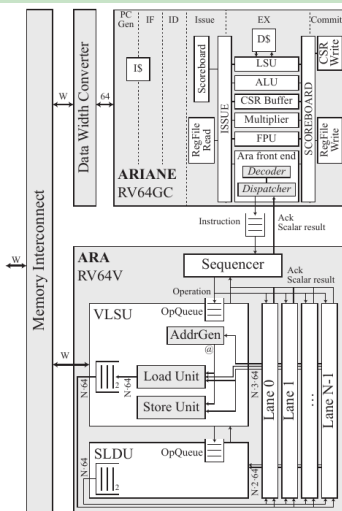
Rejestry wektorowe RISC-V

- 32 rejestry wektorowe (v0-v31).
- Ponad 250 nowych instrukcji.
- CSRy: vl, vtype, vstart, vxrm, vxsat.
- Stałe zdefiniowane na etapie implementacji: VLEN, ELEN.
- Parametry: EEW, LMUL.
- Wykonanie predykatywne.
- Wsparcie dla optymalizacji out-of-order (ustawianie 1 w zamaskowanych rejestrach) (undisturbed/ignored zamaskowane/ogonowe).
- Operacje redukcji.
- Operacje zmieniające szerokość elementów.

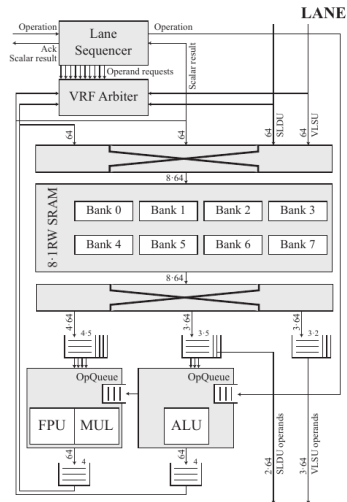
Obserwacja

Dla VLEN=4096 i LMUL=8 jeden rejestr ma 4KB.

Ara



(a) Block diagram of an Ara instance with N parallel lanes. Ara receives its commands from Ariane, a RV64GC scalar core. The vector unit has a main sequencer; N parallel lanes; a Slide Unit (SLDU); and a Vector Load/Store Unit (VLSU). The memory interface is W bit wide.



(b) Block diagram of one lane of Ara. It contains a lane sequencer (handling up to 8 vector instructions); a 16 KiB vector register file; ten operand queues; an integer Arithmetic Logic Unit (ALU); an integer multiplier (MUL); and a Floating Point Unit (FPU).

Ara

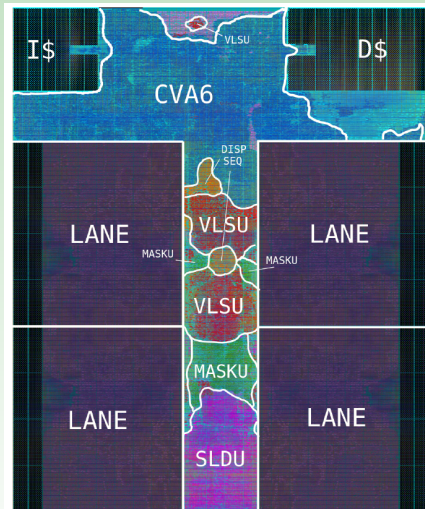


Figure 4. Physical implementation of the full system. The lane is implemented and enclosed in a macro and then placed on the die. The system input and output are at the top of the die (AXI interface).

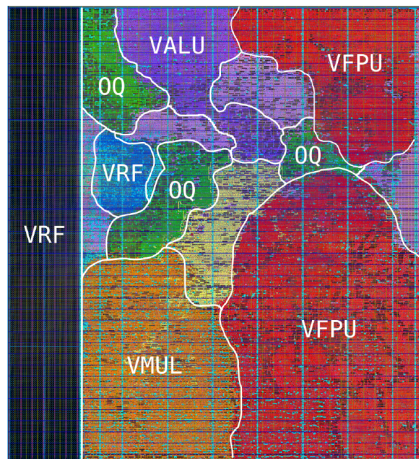


Figure 5. Physical implementation of a Lane. Modules without a label: lane sequencer, operand requesters (close to the VRF), VDIV and control logic for VMUL and VFPU (in the middle).

CVA6 stall

Table 1. Early State Space Exploration Results for Different VRF Configurations

Clock Period: 800 ps (1.25 GHz)					
Configuration	Ports	Cell Type	Area (μm^2)	Slack SS ^a /TT ^b (ps)	Power (mW)
5×2 kB	1RW	SRAM	21,005.726	0/193	25.2674
1×10 kB	3R2W	Latch	338,725.864	0/0	119.572
1×8 kB	3R2W	Latch	273,288.375	0/0	107.601
1×10 kB	1R2W	Latch	239,603.614	0/0	98.6253
1×8 kB	1R2W	Latch	184,466.954	0/0	78.5065
5×2 kB	1R1W	SRAM	35,763.728	-429/-68	32.6354

^aSlow corner conditions (SS/0.72V/125°C).

^bTypical corner conditions (TT/0.80V/25°C).

Note: 1RW = shared read/write port; 1R1W = one read port and one write port; 1R2W = one read port and two write ports; 3R2W = three read ports and two write ports.

Źródło: Minervini i in. 2023

Vitruvius+

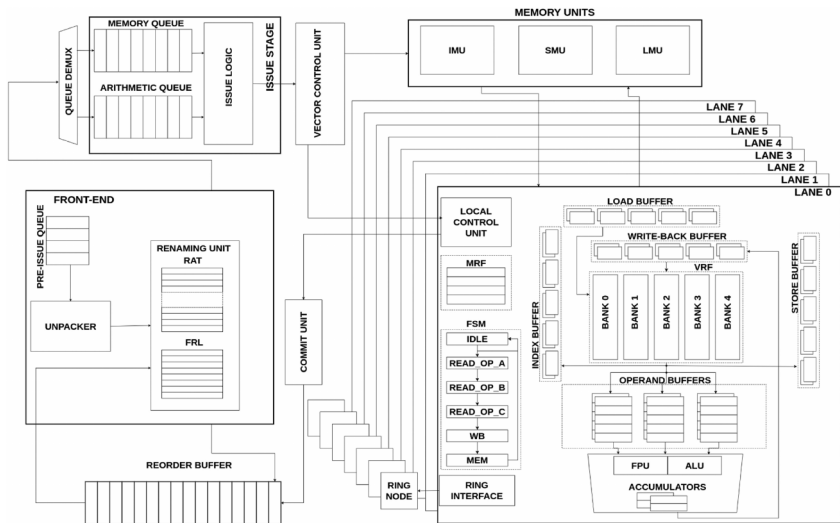
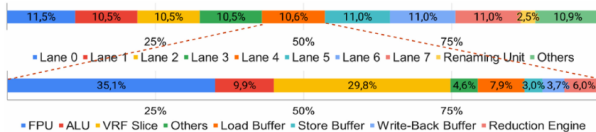


Fig. 3. Vitruvius+ top-level block diagram for the eight-lane configuration.

Table 3. Percentage of Vector-Vector Moves over the Total Arithmetic Instructions for Some of the Target Benchmarks

Benchmark	Total Arithmetic Instructions	Total vmv.v.v Instructions	Percentage
Jacobi-2D	20,402	6,163	30.2%
Streamcluster	745,236	165,608	28.6%
LavaMD	37,376	2,048	5.5%
Blackscholes	672,000	9,600	1.4%
MMUL	65,793	256	0.4%

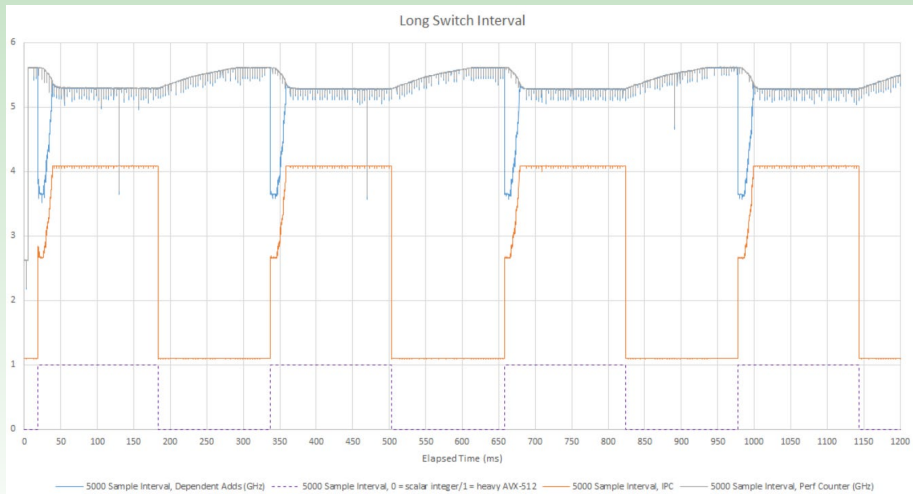
Źródło: Minervini i in. 2023



Problemy projektowe

- Czy Vector Frontend powinien być połączony z frontendem skalarnym, czy też nie (np. Ara)?
- Dostęp nawet do 5 rejestrów na raz (masked multiply-add)
- W jaki sposób obsługiwać chaining (i czy w ogóle)?
- Jak połączyć fakt, że v0 czasami zwykłym rejestrem, a czasami maską (więc potrzebuje dostępu do innych taśm)?
- Jak poradzić sobie z faktem, że instrukcje zmieniające szerokość elementów, mogą robić dostęp do innych taśm?
- Rejestr wektorowy może przechowywać skalar używany w wielu taśmach.

Zen5 – częstotliwość a IPC

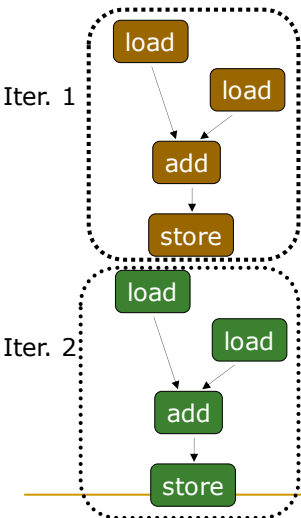


Źródło: <https://chipsandcheese.com/p/zen-5s-avx-512-frequency-behavior>

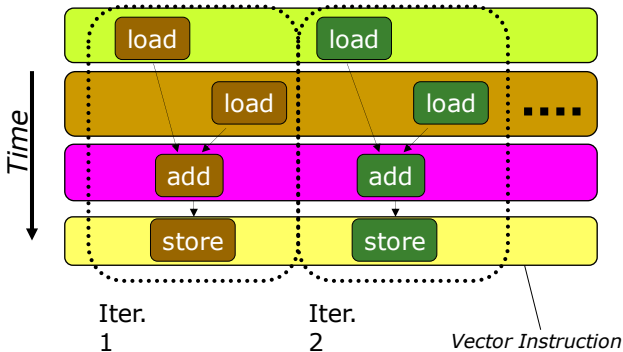
Automatic Code Vectorization

```
for (i=0; i < N; i++)  
  C[i] = A[i] + B[i];
```

Scalar Sequential Code



Vectorized Code



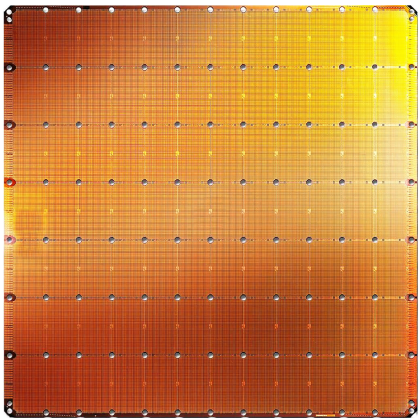
Vectorization is a compile-time reordering of operation sequencing
⇒ requires extensive loop dependence analysis

Vector/SIMD Processing Summary

- Vector/SIMD machines are good at exploiting **regular data-level parallelism**
 - Same operation performed on many data elements
 - Improve performance, simplify design (no intra-vector dependencies)
- **Performance improvement limited by vectorizability** of code
 - Scalar operations limit vector machine performance
 - Remember **Amdahl's Law**
 - CRAY-1 was the fastest SCALAR machine at its time!
- Many existing ISAs include (vector-like) SIMD operations
 - Intel MMX/SSEn/AVX, PowerPC AltiVec, ARM Advanced SIMD

SIMD Operations in Modern (Machine Learning) Accelerators

Cerebras's Wafer Scale Engine (2019)



Cerebras WSE

1.2 Trillion transistors
46,225 mm²

- The largest ML accelerator chip (2019)
- 400,000 cores



Largest GPU

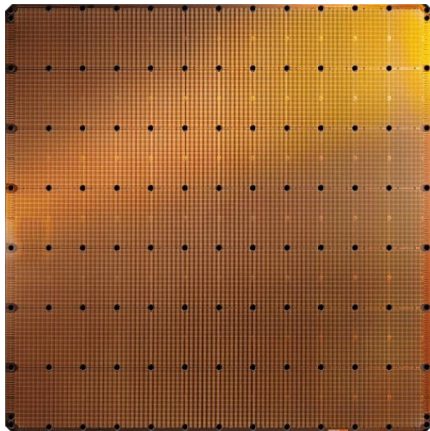
21.1 Billion transistors
815 mm²

NVIDIA TITAN V

<https://www.anandtech.com/show/14758/hot-chips-31-live-blogs-cerebras-wafer-scale-deep-learning>

<https://www.cerebras.net/cerebras-wafer-scale-engine-why-we-need-big-chips-for-deep-learning/>

Cerebras's Wafer Scale Engine-2 (2021)



Cerebras WSE-2
2.6 Trillion transistors
46,225 mm²

- The largest ML accelerator chip (2021)
- 850,000 cores



Largest GPU
54.2 Billion transistors
826 mm²

NVIDIA Ampere GA100

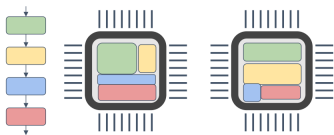
<https://www.anandtech.com/show/14758/hot-chips-31-live-blogs-cerebras-wafer-scale-deep-learning>

<https://www.cerebras.net/cerebras-wafer-scale-engine-why-we-need-big-chips-for-deep-learning/>

Size, Place, and Route in Cerebras's WSE

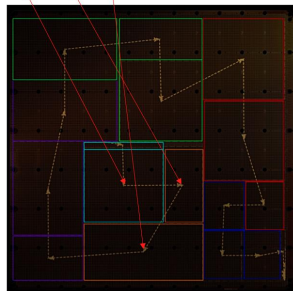
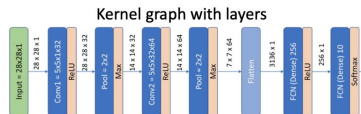
- Neural network mapping onto the whole wafer is a challenge

Multiple possible mappings



Different dies of the wafer work on different layers of the neural network: **MIMD** machine

An example mapping



Layers mapped on Wafer Scale Engine

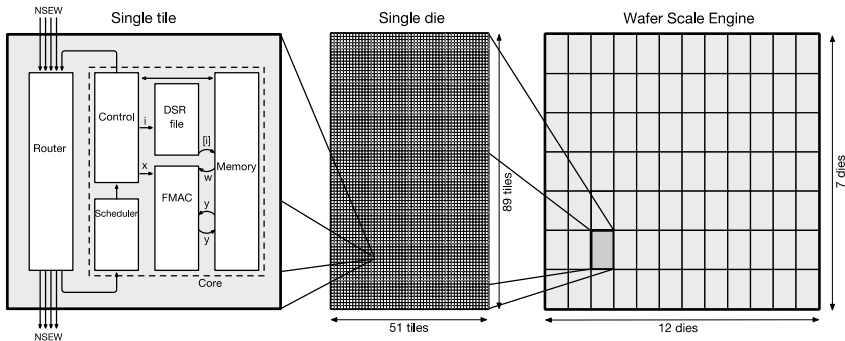
Recall: Flynn's Taxonomy of Computers

- Mike Flynn, “**Very High-Speed Computing Systems**,” Proc. of IEEE, 1966
- **SISD**: Single instruction operates on single data element
- **SIMD**: Single instruction operates on multiple data elements
 - Array processor
 - Vector processor
- **MISD**: Multiple instructions operate on single data element
 - Closest form: systolic array processor, streaming processor
- **MIMD**: Multiple instructions operate on multiple data elements (multiple instruction streams)
 - Multiprocessor
 - Multithreaded processor

A MIMD Machine with SIMD Processors (I)

■ MIMD machine

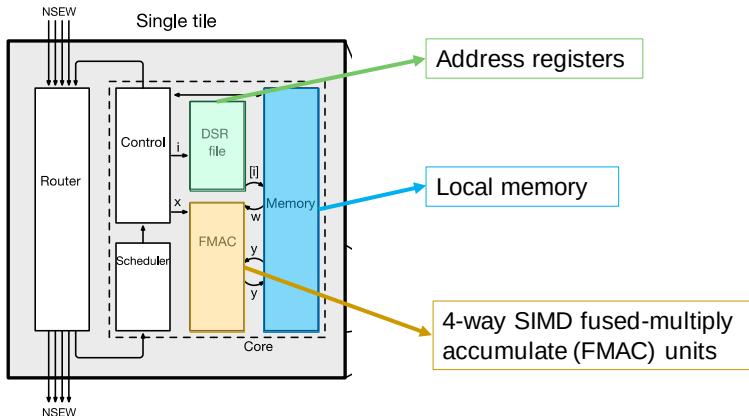
- ❑ Distributed memory (no shared memory)
- ❑ 2D-mesh interconnection fabric



A MIMD Machine with SIMD Processors (II)

■ SIMD processors

- ❑ 4-way SIMD for 16-bit floating point operands
- ❑ 48 KB of local SRAM





Cavalcante, Matheus i in. (2020). “Ara: A 1-GHz+ Scalable and Energy-Efficient RISC-V Vector Processor With Multiprecision Floating-Point Support in 22-nm FD-SOI”. *W: IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 28.2, s. 530–543. DOI: 10.1109/TVLSI.2019.2950087.



Minervini, Francesco i in. (mar. 2023). “Vitruvius+: An Area-Efficient RISC-V Decoupled Vector Coprocessor for High Performance Computing Applications”. *W: ACM Trans. Archit. Code Optim.* 20.2. ISSN: 1544-3566. DOI: 10.1145/3575861. URL: <https://doi.org/10.1145/3575861>.



Perotti, Matteo i in. (lip. 2022). “A “New Ara” for Vector Computing: An Open Source Highly Efficient RISC-V V 1.0 Vector Processor Design”. *W: 2022 IEEE 33rd International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE. DOI: 10.1109/asap54787.2022.00017. URL: <https://doi.org/10.1109%2Fasap54787.2022.00017>.